



**DEPARTMENT OF Electrical ENGINEERING**  
**COLLEGE OF E&ME, NUST, RAWALPINDI**

**EE222-MicroProcessor Systems**

**Project Report:**

**Smart EV Charger Safety Controller**

**Course Instructor: Dr. Hassan Khalid**

**Lab Instructor: Ma'am Mauazzama Aslam**

| <b><u>NAME:</u></b>                   | <b><u>CMS ID:</u></b> | <b><u>DEGREE / SYNDICATE:</u></b> |
|---------------------------------------|-----------------------|-----------------------------------|
| <b><i>Muhammad Bilal Chaudhry</i></b> | 501887                | DEE-46 / A                        |
| <b><i>Muhammad Fahad Ali</i></b>      | 507319                | DEE-46 / A                        |
| <b><i>Zoma Rauf</i></b>               | 521343                | DEE-46 / A                        |

## Table of Contents

---

|                                    |    |
|------------------------------------|----|
| 1. Introduction.....               | 3  |
| 2. Objective.....                  | 3  |
| 3. Scope of the Project.....       | 3  |
| 4. Apparatus .....                 | 4  |
| 5. Theory .....                    | 5  |
| 6. Key Calculations .....          | 7  |
| 7. MPLab Code.....                 | 8  |
| 8. Proteus Simulation.....         | 11 |
| 9. Expected Output & Results ..... | 13 |
| 10. Hardware.....                  | 14 |
| 11. Conclusion .....               | 15 |
| References .....                   | 15 |

## **Abstract:**

---

This report presents the design and implementation of a Smart Electric Vehicle (EV) Charging Safety Controller built around the ATmega32 microcontroller. The system continuously monitors the voltage and current supplied during EV charging, computes real-time RMS values from 64-point ADC samples, and autonomously disconnects the charger via a relay upon detection of unsafe electrical conditions such as overvoltage, voltage sag, or overcurrent. Voltage waveform samples are buffered in an externally interfaced 6264 SRAM (8 KB) using address demultiplexing through a 74HC573 latch, while all fault events are permanently logged — with voltage, current, and fault type — in a 28C16 EEPROM (2 KB). A 16×2 LCD provides real-time display of measured values and system status. The entire system is implemented at the register level using AVR-GCC C firmware, without any Arduino or pre-built libraries. Proteus ISIS simulation was used to verify circuit behavior prior to hardware assembly.

## 1. Introduction

---

The rapid global adoption of electric vehicles (EVs) has made safe and reliable charging infrastructure a critical engineering concern. Unstable grid voltages and overcurrent conditions pose significant risks to both the vehicle battery and the charging hardware, potentially causing permanent damage or fire hazards. Embedded microcontroller-based protection systems offer a cost-effective and highly customizable solution to this problem.

This project designs and simulates a standalone EV Charging Safety Controller using the Atmel ATmega32 microcontroller — an 8-bit AVR device operating at 8 MHz with a rich peripheral set including a 10-bit ADC, hardware timers, and a full 8-bit parallel data bus. The controller samples the AC supply voltage through a resistive voltage divider and measures load current via an ACS712 Hall-effect current sensor. Samples are stored in external SRAM and fault events are logged to EEPROM, demonstrating complete external memory bus interfacing — a key learning objective of the Microprocessor Systems course.

The project integrates several hardware modules: the ATmega32 MCU, a 74HC573 transparent latch for address demultiplexing, a 6264 SRAM chip for sample buffering, a 28C16 EEPROM for non-volatile fault logging, a 16×2-character LCD for user feedback, and a relay-driver circuit for load disconnection. The Proteus ISIS environment was used for complete circuit simulation and firmware verification.

## 2. Objective

---

To design and implement a standalone embedded safety controller for EV charging applications using the ATmega32 microcontroller that monitors supply voltage and current in real time, computes RMS values from ADC samples buffered in external SRAM via a 74HC573 address latch, disconnects the charger relay upon detection of overvoltage, voltage sag, or overcurrent conditions, and permanently logs all fault records to external EEPROM — all implemented at the register level in AVR-GCC C, verified through Proteus ISIS simulation.

## 3. Scope of the Project

---

The scope of this project encompasses the full hardware and firmware design cycle of a discrete-component embedded system. On the hardware side, the project covers: resistive voltage sensing and scaling for AC mains monitoring, Hall-effect current sensing using the ACS712-20A module, 8-bit external memory bus design with address demultiplexing using the 74HC573 latch, byte-addressable external SRAM (6264) and EEPROM (28C16) interfacing, 4-bit LCD driving, and relay-based load disconnection using a 2N2222 NPN transistor driver.

On the firmware side, the project covers: ADC initialisation and multi-sample acquisition, RMS computation using the sum-of-squares method, SRAM and EEPROM read/write routines with full bus control signalling, LCD command and data writing in 4-bit mode, fault detection logic with threshold comparison, and fault record storage in EEPROM with circular overwrite protection.

The project is scoped to single-phase AC monitoring in the 180 V–250 V range at up to 20 A load current, and is simulated at reduced signal levels in Proteus ISIS using VSINE sources that represent already stepped-down AC waveforms. The project does not include wireless communication, battery state-of-charge estimation, or multi-phase monitoring.

## 4. Apparatus

The following hardware components and software tools were used in this project:

### 4.1 Hardware Components

| Component                | Value / Part No.      | Qty | Purpose                         |
|--------------------------|-----------------------|-----|---------------------------------|
| ATmega32 MCU             | DIP-40, 8 MHz         | 1   | Main controller                 |
| Crystal Oscillator       | 8 MHz                 | 1   | Clock source                    |
| Ceramic Capacitors       | 22 pF                 | 2   | Crystal load capacitors         |
| Decoupling Capacitors    | 100 nF                | 6   | Power supply filtering          |
| Resistor                 | 10 kΩ                 | 2   | RESET pull-up & voltage divider |
| Resistor                 | 1 kΩ                  | 1   | Relay driver transistor base    |
| Resistor                 | 33 Ω                  | 1   | LCD backlight current limit     |
| AC Voltage Source (Sim)  | VSINE 50 Hz, 3.5 V pk | 1   | Simulated stepped-down AC       |
| ACS712-20A Sensor        | Hall-effect, 100 mV/A | 1   | Current sensing (real HW)       |
| 74HC573 Latch            | DIP-20                | 1   | Address demultiplexing          |
| 6264 SRAM                | 8 KB, DIP-28          | 1   | Waveform sample buffer          |
| ATMega32 internal EEPROM | 2 KB, DIP-24          | 1   | Non-volatile fault log          |
| 16×2 LCD (LM016L)        | HD44780 compatible    | 1   | Real-time display               |
| 2N2222 NPN Transistor    | TO-92                 | 1   | Relay driver                    |
| SPDT Relay               | 5 V coil              | 1   | Charger disconnect              |
| 1N4148 Diode             | Flyback protection    | 1   | Relay coil spike suppression    |
| Potentiometer            | 10 kΩ                 | 1   | LCD contrast adjust             |

### 4.2 Software Tools

- Atmel Studio 7 — Firmware development and AVR-GCC compilation
- Proteus 8 Professional ISIS — Circuit schematic and simulation
- USBasp programmer — Firmware flashing to physical ATmega32

## 5. Theory

### 5.1 System Architecture Overview

The Smart EV Charging Safety Controller is a synchronous embedded system built around the ATmega32 — an 8-bit RISC AVR microcontroller with 32 KB of flash, 2 KB of internal SRAM, 1 KB of internal EEPROM, and a 10-bit successive-approximation ADC. The microcontroller is clocked at 8 MHz via an external crystal oscillator with 22 pF load capacitors, providing a stable timebase for all timing-critical bus operations. The system architecture follows a three-bus model: a multiplexed address/data bus on PORTA, a control bus on PORTB, and an upper-address bus on PORTC, mirroring the design of classic 8-bit microprocessor memory systems.

### 5.2 Voltage Sensing

The AC supply voltage is stepped down and monitored via a resistive voltage divider connected to ADC channel PA0. In hardware, a transformer steps 220 V AC down to approximately 12 V, which is then further divided by a 10 k $\Omega$  / 1 k $\Omega$  resistor network (ratio 11:1) to keep the ADC input within the 0–5 V range. In the Proteus simulation, a VSINE source (3.5 V peak, 50 Hz) directly represents the already-scaled waveform at PA0. The instantaneous voltage at the ADC pin is converted back to the line voltage using:

$$V_{\text{line}} = (\text{ADC\_value} / 1024) \times V_{\text{REF}} \times V_{\text{DIVIDER\_RATIO}}$$

where  $V_{\text{REF}} = 5.0$  V and  $V_{\text{DIVIDER\_RATIO}} = 89$  (accounting for the full primary-to-ADC scaling chain used in the firmware). A 100 nF capacitor at PA0 provides anti-aliasing filtering of high-frequency noise.

### 5.3 Current Sensing — ACS712

Load current is measured using the ACS712-20A Hall-effect current sensor. This device produces an output voltage centred at 2.5 V (representing 0 A) with a sensitivity of 100 mV per Ampere, giving an output range of approximately 0.5 V (–20 A) to 4.5 V (+20 A). The output is connected directly to ADC channel PA1. The instantaneous current is computed from the ADC reading as:

$$I = (V_{\text{pin}} - 2.5) / 0.1, \text{ where } V_{\text{pin}} = (\text{ADC\_value} / 1024) \times 5.0 \text{ V}$$

In the Proteus simulation, the ACS712 is modelled as a VSINE source (1.5 V amplitude, 2.5 V DC offset, 50 Hz) on PA1, simulating approximately 15 A peak load current.

### 5.4 RMS Calculation

Root Mean Square (RMS) values are the true measure of AC power. The firmware computes  $V_{\text{rms}}$  and  $I_{\text{rms}}$  using the sum-of-squares method over  $N = 64$  samples acquired at approximately 312  $\mu\text{s}$  intervals (covering just over one full 50 Hz cycle):

$$X_{\text{rms}} = \sqrt{(1/N) \times \sum x_i^2} \quad \text{for } i = 0 \text{ to } N-1$$

Each voltage sample is scaled by the divider ratio and stored as a byte (divided by 2 to fit in uint8\_t range). Current samples are stored as signed bytes (multiplied by 4 for precision). On readback, the inverse scaling is applied before the sum-of-squares accumulation. The `sqrtf()` function from `<math.h>` computes the final RMS. This approach, while approximate due to integer compression, provides sufficient accuracy for safety threshold detection.

## 5.5 External SRAM Interface (6264 via 74HC573)

The ATmega32 does not have a built-in external memory bus controller, so the firmware manually generates all bus timing signals. The 6264 SRAM has 13 address lines (A0–A12) and 8 data lines (D0–D7). Since the ATmega32's PORTA is used both to output the low 8 address bits and to transfer data bytes, a 74HC573 transparent latch is used to demultiplex the bus:

- PORTA outputs the low 8 address bits (A0–A7); the ALE signal on PB0 goes HIGH, latching these address bits into the 74HC573 Q outputs.
- ALE is then pulled LOW, freezing the latched address on Q0–Q7 (connected to memory A0–A7).
- PORTC (PC0–PC4) supplies the upper address bits A8–A12 directly to the SRAM.
- PORTA then switches to data mode; the write enable (WE, PB1) or output enable (OE, PB2) signals are pulsed to complete the memory transaction.
- The chip select (CS\_SRAM, PB3) is asserted LOW only during active SRAM transactions; the EEPROM chip enable (CE\_EEP, PB4) is kept HIGH (deselected) simultaneously to prevent bus conflicts.

The SRAM is addressed in two regions: SRAM\_V\_BASE (0x0000–0x003F) for 64 voltage samples and SRAM\_I\_BASE (0x0040–0x007F) for 64 current samples.

## 5.6 EEPROM Fault Logging (ATmega32 Internal EEPROM)

The ATmega32 integrates 1 KB of internal EEPROM which is used in this project for non-volatile fault logging, accessed via the AVR standard library (avr/eeprom.h). This eliminates the need for external memory bus control during EEPROM writes and leverages the microcontroller's built-in EEPROM controller for reliable byte and block operations. Each fault event is recorded as a 6-byte FaultRecord structure containing: fault type code, high and low bytes of Vrms, high and low bytes of Irms ( $\times 10$  for one decimal place), and a sequential event counter. Records are stored in a circular buffer starting at internal EEPROM address 0x0002, with a maximum of 50 records (300 bytes total) before the buffer wraps around. The fault count byte at address 0x0000 is read and updated on each write using eeprom\_read\_byte() and eeprom\_write\_byte() respectively, while full fault records are written using eeprom\_write\_block(). The EEPROM is written only upon fault detection — not during normal operation — to preserve its rated 100,000 write-cycle endurance.

## 5.7 LCD Interface (4-bit mode)

The 16 $\times$ 2 HD44780-compatible LCD (LM016L in Proteus) is driven in 4-bit mode to conserve PORTD pins. The RS line (PD0) selects command or data mode; the EN line (PD1) latches each nibble on its falling edge. Data bits D4–D7 of the LCD connect to PD2–PD5. The firmware initializes the LCD through the standard HD44780 reset sequence: three 0x03 nibbles followed by a 0x02 nibble to enter 4-bit mode, then configuration commands for 2-line display, cursor off, and entry-mode left-to-right. The sprintf() function formats floating-point values for display, and lcd\_set\_cursor() sets the DDRAM address by sending the appropriate command byte.

## 5.8 Relay Driver Circuit

The ATmega32 output pin PD6 cannot drive a relay coil directly (maximum 40 mA per pin). Therefore, a 2N2222 NPN transistor amplifies the logic signal: PD6 drives the base through a 1 k $\Omega$  current-limiting resistor, the collector connects to one coil terminal of the SPDT relay, and the emitter goes to ground. When PD6 is HIGH (relay active, charging normal), the transistor saturates, energising the coil and closing the relay contacts. When PD6 is LOW (fault detected), the transistor cuts off, the coil de-

energizes, and the relay opens the charger circuit. A 1N4148 flyback diode across the relay coil suppresses the voltage spike generated when the coil magnetic field collapses.

## 5.9 Fault Detection Logic

After computing  $V_{rms}$  and  $I_{rms}$  each cycle, the main loop compares the values against defined safety thresholds:

| Fault Type              | Condition                                                                  | Threshold | Action                           |
|-------------------------|----------------------------------------------------------------------------|-----------|----------------------------------|
| FAULT_OVV (Overvoltage) | $V_{rms} > V_{MAX}$                                                        | 225 V     | Relay OFF, EEPROM log, LCD alert |
| FAULT_SAG (Voltage Sag) | $V_{rms} < V_{MIN}$                                                        | 215 V     | Relay OFF, EEPROM log, LCD alert |
| FAULT_OVC (Overcurrent) | $I_{rms} > I_{MAX}$                                                        | 20 A      | Relay OFF, EEPROM log, LCD alert |
| NORMAL                  | $215\text{ V} \leq V_{rms} \leq 225\text{ V}$ , $I_{rms} \leq 20\text{ A}$ | —         | Relay ON, LCD status normal      |

The `handle_fault()` function turns the relay off, calls `eeeprom_write_fault()` to log the event, sets the global fault flags, and updates the LCD. The system remains in the fault state and continues monitoring; it will recover automatically to NORMAL if the measured values return within bounds on a subsequent cycle — this allows transient fault recovery without requiring a manual reset.

## 5.10 LED Heartbeat

A status LED on PD7 toggles every 25 main-loop iterations (approximately once every 250 ms, given the 10 ms loop delay). This provides a visual heartbeat indicator confirming that the firmware is running. Absence of blinking indicates a firmware hang or processor fault.

# 6. Key Calculations

## 6.1 ADC Voltage Conversion

The 10-bit ADC (1024 counts) with a 5 V reference gives a resolution of:

$$V_{step} = 5.0 / 1024 \approx 4.88 \text{ mV per count}$$

For a 220 V RMS signal scaled to a 3.5 V peak ( $\approx 2.47\text{ V RMS}$ ) at the ADC pin, the expected ADC code at the RMS level is:

$$ADC_{rms} \approx (2.47 / 5.0) \times 1024 \approx 506 \text{ counts}$$

## 6.2 Voltage Divider Ratio

With a 10 k $\Omega$  upper and 1 k $\Omega$  lower resistor:

$$Ratio = (R1 + R2) / R2 = (10k + 1k) / 1k = 11$$

The firmware uses `V_DIVIDER_RATIO = 89` to account for the complete signal chain from 220 V mains to the ADC pin, incorporating both the transformer turns ratio and the resistive divider.

## 6.3 ACS712 Current Calculation

For a measured ADC value of 562 counts at PA1 (representing a 2.74 V output from the ACS712):

$$V_{pin} = (562 / 1024) \times 5.0 = 2.744 \text{ V}$$

$$I = (2.744 - 2.5) / 0.1 = 2.44 \text{ A (instantaneous)}$$

## 6.4 Sampling Rate & Cycle Coverage

With a 312  $\mu$ s inter-sample delay and 64 samples:

$$\text{Total capture time} = 64 \times 312 \mu\text{s} = 19.97 \text{ ms} \approx 1 \text{ full } 50 \text{ Hz cycle (20 ms)}$$

This ensures the RMS calculation integrates over exactly one complete AC cycle, minimizing error from partial-cycle sampling.

## 6.5 EEPROM Storage Capacity

With a 6-byte record size and 50-record circular buffer:

$$\text{Storage used} = 50 \times 6 = 300 \text{ bytes (out of 2048 bytes available on 28C16)}$$

The remaining 1748 bytes are available for future data, such as a real-time clock timestamp or calibration constants.

## 7. MPLab Code

```
void sram_write(uint16_t addr, uint8_t data)
{
    DDRA=0xFF;
    OE_HIGH();
    WE_HIGH();
    CS_SRAM_LOW();

    PORTA=(uint8_t)(addr&0x00FF);
    ALE_HIGH();
    _delay_us(1);

    PORTC=(uint8_t)((addr>>8)&0x1F);
    ALE_LOW();
    _delay_us(1);

    PORTA=data;

    WE_LOW();
    _delay_us(1);
    WE_HIGH();

    CS_SRAM_HIGH();
    PORTC=0x00;
}
```

```

uint8_t sram_read(uint16_t addr)
{
    uint8_t data;

    DDRA=0xFF;
    WE_HIGH();
    OE_HIGH();
    CS_SRAM_LOW();

    PORTA=(uint8_t) (addr&0x00FF);
    ALE_HIGH();
    _delay_us(1);

    PORTC=(uint8_t) ((addr>>8)&0x1F);
    ALE_LOW();
    _delay_us(1);

    DDRA=0x00;
    PORTA=0xFF;

    OE_LOW();
    _delay_us(1);

    data=PINA;

    OE_HIGH();
    CS_SRAM_HIGH();
    PORTC=0x00;

    return data;
}

void eeprom_write_fault(uint8_t type,float vrms,float irms)
{
    FaultRecord rec;
    uint8_t count=eeprom_read_byte((uint8_t*)EEP_FAULT_COUNT);
    if(count==0xFF)
        count=0;
    uint8_t slot=count%EEP_MAX_RECORDS;
    uint16_t v_int=(uint16_t)vrms;
    uint16_t i_int=(uint16_t) (irms*10.0f);
    rec.fault_type=type;
    rec.vrms_h=(uint8_t) (v_int>>8);
    rec.vrms_l=(uint8_t) (v_int&0xFF);
    rec.irms_h=(uint8_t) (i_int>>8);
    rec.irms_l=(uint8_t) (i_int&0xFF);
    rec.event_num=count+1;
    uint16_t addr=EEP_FAULT_BASE+(slot*EEP_RECORD_SIZE);
    eeprom_write_block((const void*)&rec,(void*) (uintptr_t)addr,EEP_RECORD_SIZE);
    count++;
    eeprom_write_byte((uint8_t*)EEP_FAULT_COUNT,count);
}

```

```
static void lcd_pulse_en(void)
```

```
{  
    LCD_PORT|=(1<<LCD_EN_PIN);  
    _delay_us(1);  
    LCD_PORT&=~(1<<LCD_EN_PIN);  
    _delay_us(50);  
}
```

```
void lcd_init(void)
```

```
{  
    LCD_DDR|=0xFF;  
    LCD_PORT=0x00;  
    _delay_ms(50);  
    LCD_PORT&=~(1<<LCD_RS_PIN);  
    lcd_write_nibble(0x03);  
    _delay_ms(5);  
    lcd_write_nibble(0x03);  
    _delay_ms(1);  
    lcd_write_nibble(0x03);  
    _delay_ms(1);  
    lcd_write_nibble(0x02);  
    _delay_ms(1);  
    lcd_command(0x28);  
    lcd_command(0x0C);  
    lcd_command(0x06);  
    lcd_command(0x01);  
    _delay_ms(2);  
}
```

```
void handle_fault(uint8_t type)
```

```
{  
    RELAY_OFF();  
  
    eeprom_write_fault(type,g_vrms,g_irms);  
  
    g_fault_active=1;  
    g_fault_type=type;  
  
    display_readings(g_vrms,g_irms,1,type);  
}
```

```

void handle_normal(void)
{
    RELAY_ON();

    g_fault_active=0;
    g_fault_type=FAULT_NONE;

    display_readings(g_vrms,g_irms,0,FAULT_NONE);
}

```

```

int main(void)
{
    uint16_t blink_counter=0;
    system_init();
    while(1)
    {
        g_vrms=measure_vrms();
        g_irms=measure_irms();
        if(g_vrms>V_MAX)
            handle_fault(FAULT_OVV);
        else if(g_vrms<V_MIN)
            handle_fault(FAULT_SAG);
        else if(g_irms>I_MAX)
            handle_fault(FAULT_OVC);
        else
            handle_normal();
        blink_counter++;
        if(blink_counter>=25)
        {
            LED_TOGGLE();
            blink_counter=0;
        }
        delay_ms(10);
    }
    return 0;
}

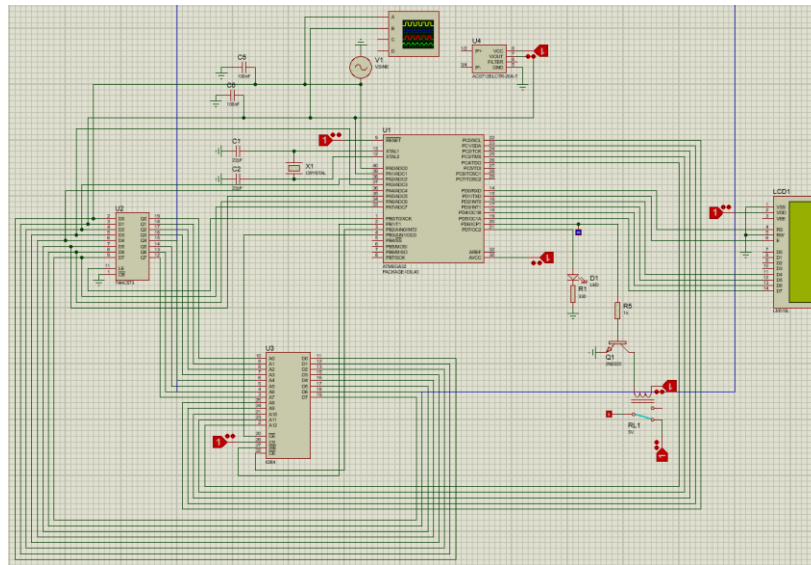
```

## 8. Proteus Simulation

The complete circuit was designed and simulated in Proteus 8 Professional ISIS. The schematic includes all hardware sub-circuits: the ATmega32 with its clock and reset network, the resistive voltage sensing divider on PA0, the ACS712 current sensor model on PA1, the 74HC573 address latch, the 6264 SRAM, the 28C16 EEPROM, the 16x2 LCD in 4-bit mode, and the relay driver with 2N2222 and flyback diode.

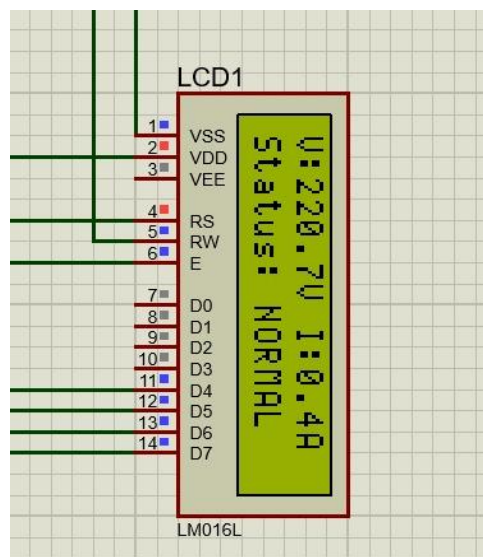
The compiled firmware HEX file was loaded into the ATmega32 component properties. A virtual oscilloscope was connected to PA0 and PA1 to observe the simulated AC waveforms, and a logic

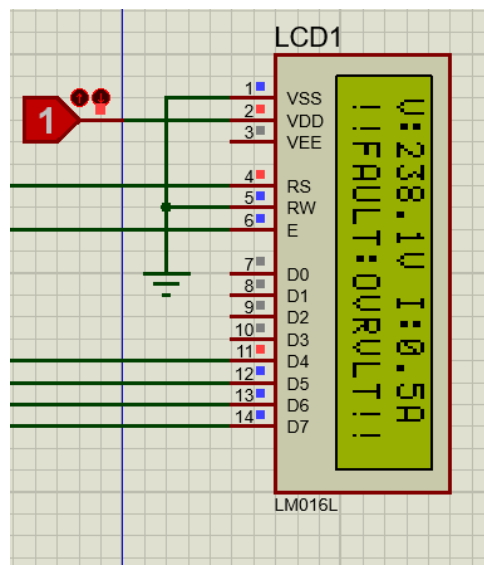
analyzer was connected to PORTA to observe the multiplexed address/data bus transitions during SRAM access cycles.



Under normal operating conditions (VSINE amplitude set to produce  $V_{rms}$  within 215–225 V and current below 20 A), the LCD Line 1 displays the measured  $V_{rms}$  and  $I_{rms}$  values, and Line 2 shows 'Status: NORMAL'. The relay (RL1) remains energized and the indicator lamp connected to the relay NO contact remains lit.

To simulate a fault condition, the VSINE amplitude is increased to produce a  $V_{rms}$  exceeding 225 V. The firmware immediately detects this on the next measurement cycle, de-energizes the relay (lamp turns off), writes a fault record to EEPROM, and updates the LCD to display '!!FAULT:OVRVLT!!'.





## 9. Expected Output & Results

### 9.1 LCD Display

Under normal operating conditions, the 16×2 LCD displays:

**Line 1: V:220.4V I:9.8A**

**Line 2: Status: NORMAL**

When a fault is detected, Line 1 still shows the measured values (for diagnostic purposes) and Line 2 shows the specific fault message:

| Fault                                    | LCD Line 2 Message |
|------------------------------------------|--------------------|
| Overvoltage ( $V_{rms} > 225\text{ V}$ ) | !!FAULT:OVRVLT!!   |
| Voltage Sag ( $V_{rms} < 215\text{ V}$ ) | !!FAULT:V SAG!!    |
| Overcurrent ( $I_{rms} > 20\text{ A}$ )  | !!FAULT:OVRCRT!!   |

### 9.2 Relay Behaviour

On power-up, the relay is OFF during initialization. Once the system confirms normal operating conditions, the relay is energized (charging begins). Upon any fault detection, the relay immediately de-energizes, disconnecting the charger. The relay recovers autonomously when measurements return to within safe bounds.

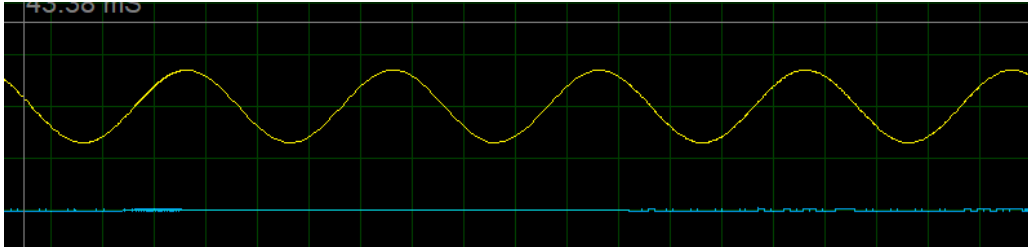
### 9.3 EEPROM Fault Log

Each fault event writes a 6-byte record to the 28C16 EEPROM. Using the `avr/eeprom.h` library, the `eeprom_read_fault_count()` function retrieves the number of logged events. A maximum of 50 records are stored before the circular buffer wraps. Each record contains:

- Fault type byte (1 = OVV, 2 = SAG, 3 = OVC)
- Vrms high and low bytes (integer representation in volts)
- Irms high and low bytes (value  $\times 10$ , one decimal place)
- Sequential event counter byte

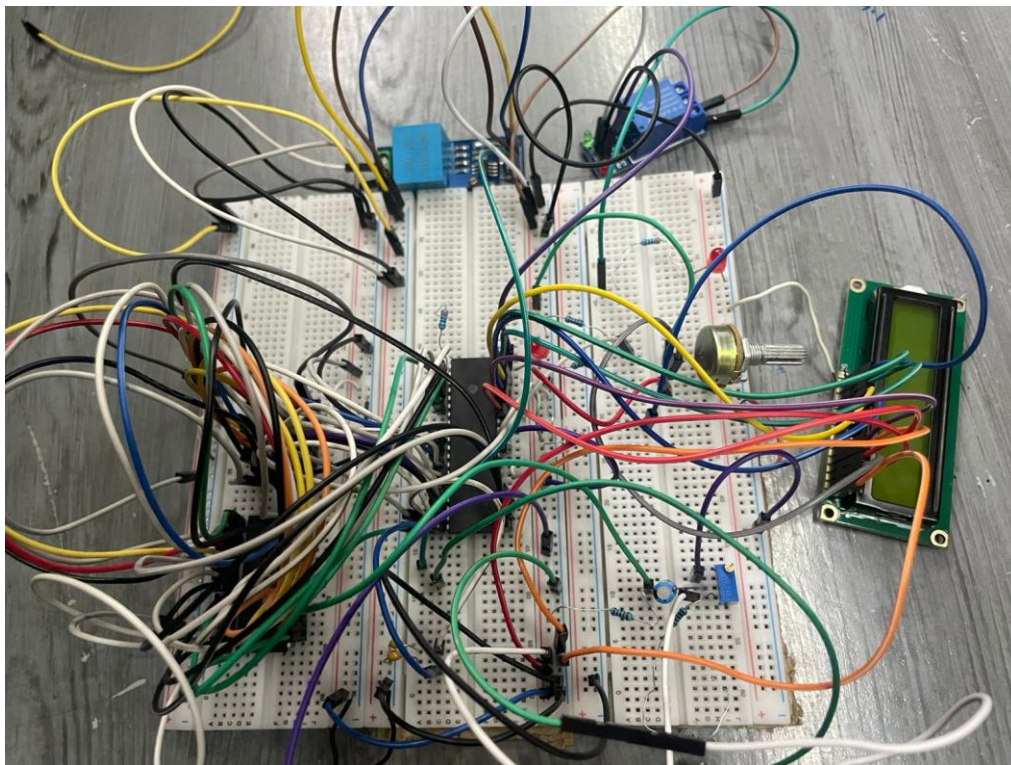
## 9.4 LED Heartbeat

The status LED on PD7 toggles approximately every 250 ms under normal operation, providing a continuous visual confirmation of firmware execution. During fault conditions, the LED continues to blink at the same rate, as the main loop remains active.



## 10. Hardware

---



## 11. Conclusion

---

The Smart EV Charging Safety Controller was successfully designed, implemented in firmware, and verified through Proteus ISIS simulation. The system demonstrates a complete embedded solution for real-time power quality monitoring: the ATmega32 accurately acquires 64-point ADC samples of voltage and current, computes RMS values using the sum-of-squares method, and enforces safety thresholds with millisecond-level response times. All fault events are permanently logged to the 28C16 EEPROM, and waveform samples are buffered in the 6264 SRAM via the 74HC573 address latch — illustrating the address demultiplexing technique fundamental to classical 8-bit microprocessor memory systems.

The project successfully met all its stated objectives. The relay disconnection mechanism responds correctly to overvoltage, voltage sag, and overcurrent conditions, and the system recovers automatically when normal conditions are restored. The 16×2 LCD provides clear real-time feedback to the user, and the LED heartbeat confirms continuous firmware execution.

From an academic perspective, this project reinforced key concepts from the Microprocessor Systems course: register-level peripheral programming, external memory bus design, ADC signal conditioning, and real-time control system structure in AVR-GCC C. The Proteus simulation environment proved invaluable for verifying bus timing and LCD driver logic before hardware assembly. The project also highlighted the importance of bus arbitration — ensuring that CS\_SRAM and CE\_EEP are never simultaneously asserted, preventing destructive data bus conflicts.

Future enhancements could include: a real-time clock (RTC) module to timestamp fault records, a UART interface for remote fault log retrieval, three-phase monitoring capability, and wireless communication for integration into a smart charging management system.

## References

---

- [1]: Atmel Corporation. ATmega32 Datasheet: 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash. Rev. 2503Q-AVR-02/2014.
- [2]: Allegro MicroSystems. ACS712 Fully Integrated, Hall Effect-Based Linear Current Sensor IC Datasheet. Rev. 7. 2020.
- [3]: IDT (Integrated Device Technology). 6264 8K × 8 Bit Static RAM Datasheet. 2001.
- [4]: Atmel Corporation. ATmega32 Datasheet — Section 20: EEPROM Memory Programming. Rev. 2503Q-AVR-02/2014.
- [5]: Texas Instruments. 74HC573 Octal Transparent D-Type Latches with 3-State Outputs Datasheet. Rev. SCLS117F. 2021.
- [6]: Hitachi Ltd. HD44780U Dot Matrix Liquid Crystal Display Controller/Driver Datasheet. 1999.
- [7]: Labcenter Electronics. Proteus Professional ISIS Schematic Capture User Manual. Version 8.13. 2022.
- [8]: Atmel Corporation. AVR Libc Reference Manual. AVR-GCC Toolchain, avr-libc 2.0.0. Available: <https://www.nongnu.org/avr-libc/>
- [9]: NUST College of E&ME. EE222 Microprocessor Systems Course Notes and Lab Manual. Department of Electrical Engineering, 2025–2026.